MapFlow

Zero-reflection, zero-dependency object mapper for .NET

AOT Compatible | Source Generator | No DI Required

Complete Guide v1.0

github.com/HancerMercede/MapFlow

Table of Contents

1. Introduction & Philosophy

MapFlow is an object-to-object mapper for .NET built on a simple premise: mapping should be fast, explicit, and have zero surprises. It was designed from the ground up to avoid the three biggest pain points of existing mappers: reflection overhead, hidden runtime behavior, and AOT incompatibility.

MapFlow provides three mapping strategies that scale with your needs: lambda-based selectors for ad-hoc projections, interface-based mapping for repeatable and testable transformations, and a Roslyn Source Generator that auto-generates mapping code at compile time for maximum performance.

The core philosophy: the developer always knows what they are mapping. No convention-based magic, no runtime profiles, no global configuration. MapFlow is a tool, not a framework.

2. Why MapFlow? The Problem Statement

Object mapping is one of the most common tasks in .NET applications, especially in layered architectures where domain entities, DTOs, view models, and commands need to be transformed between boundaries. The typical solutions have trade-offs:

Manual mapping (hand-written code)

Writing mapping code by hand gives you full control and maximum performance, but it is tedious, repetitive, and error-prone. Every new property means updating multiple mapping blocks across the codebase.

AutoMapper

The most popular mapper in the .NET ecosystem. Powerful but heavy. It uses reflection at startup to scan profiles, builds expression trees for every mapping pair, and compiles them at runtime via Reflection.Emit. This means: slow startup, high memory consumption, non-trivial debugging when mappings fail, and complete incompatibility with Native AOT.

Mapster

Faster than AutoMapper in benchmarks, but shares the same fundamental architecture: reflection-based configuration, runtime expression compilation, and no AOT support. It does allow code generation but as an afterthought.

MapFlow's approach

MapFlow takes a different path: instead of trying to be 'AutoMapper but faster', it entirely removes the need for runtime reflection and code generation. The mapping logic is either written explicitly (lambdas), implemented by the developer via interfaces, or generated at compile time by a Roslyn Source Generator. The result: zero warmup, predictable performance, and a mapper that works everywhere .NET runs.

3. Architecture Overview

MapFlow consists of two components that work together seamlessly:

MapFlow.Core (MapFlow.dll)

The runtime library. Contains the static Mapper class, MapperExtensions (fluent .Map() and .Apply() methods), PagedResult<T>, and the interfaces IMapFrom<TSource> and IMapTo<TDest>. This is the only package dependency your project needs. It targets net8.0 and has ZERO external dependencies.

MapFlow.SourceGenerator (MapFlow.SourceGenerator.dll)

A Roslyn incremental source generator that runs at compile time. It detects types implementing IMapFrom<T> or IMapTo<T> and generates the MapFrom() / MapTo() method bodies by matching property names and types. The generated code is regular C# that gets compiled as part of your project. The SG targets netstandard2.0 and ships embedded in the MapFlow NuGet package.

How they connect

When you call Mapper.Map<TSource, TDest>(source), MapFlow uses the interface constraint 'new() + IMapFrom<TSource>' to create a new instance and call MapFrom(). The Source Generator simply automates writing the MapFrom() body. There is no runtime discovery, no service locator, no DI - the compiler resolves everything.

4. Mapping Modes

MapFlow offers four distinct mapping modes, each optimized for a different scenario.

4.1 Lambda-based (Selector)

The simplest and most flexible mode. You provide a lambda expression that transforms the source into the destination. MapFlow simply invokes it.

```
// Single object
var dto = product.Map(p => new ProductDto
{
    Id = p.Id,
    Name = p.Name,
    Price = p.Price
});

// Collection
var dtos = products.Map(p => new ProductDto(p.Id, p.Name, p.Price));

// With complex logic
var dtos = products.Map(p =>
{
    var status = p.Stock > 0 ? "Available" : "Out of stock";
    return new ProductDto(p.Id, p.Name, status);
});
```

This is as fast as hand-written code because it IS hand-written code. The compiler inlines the delegate call. No reflection, no cache, no magic.

4.2 Interface-based (IMapFrom / IMapTo)

For repeatable mappings across your codebase, implement IMapFrom<TSource> or IMapTo<TDest> on your DTO/entity. MapFlow handles instantiation and dispatch.

```
public class ProductDto : IMapFrom<Product>
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;

    public void MapFrom(Product source)
    {
        Id = source.Id;
        Name = source.Name;
    }
}

ProductDto dto = Mapper.Map<Product, ProductDto>(product);
ProductDto dto2 = product.MapTo<ProductDto>();
List<ProductDto> dtos = Mapper.Map<Product, ProductDto>(products);
```

IMapTo works symmetrically for the reverse direction:

```
public class Product : IMapTo<ProductDto>
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;

    public ProductDto MapTo() => new()
    {
        Id = Id,
        Name = Name
    };
}

ProductDto dto = product.MapTo<ProductDto>();
```

4.3 Source Generator (SG)

The SG eliminates boilerplate by auto-generating MapFrom() / MapTo() at compile time. Just declare the interface and matching properties.

```
public partial class ProductDto : IMapFrom<Product>
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public decimal Price { get; set; }
}

ProductDto dto = Mapper.Map<Product, ProductDto>(product);
```

The SG matches properties by name and type. For custom logic, implement the partial method hook:

```
public partial class ProductDto : IMapFrom<Product>
{
    public int Id { get; set; }
    public string FullName { get; set; } = string.Empty;

    // SG generates: Id = source.Id;
    // You add custom mappings:
    partial void CustomMapFrom(Product source)
    {
        FullName = $"{source.FirstName} {source.LastName}";
    }
}
```

4.4 In-Place Mutation (Apply)

Apply() mutates an existing object and returns it. Perfect for update scenarios and method chaining.

```
// Mutate and return same instance
await repo.UpdateAsync(product.Apply(p =>
{
    p.Name = request.Name;
    p.Price = request.Price;
}), ct);

// Transform overload - returns new instance
var updated = product.Apply(p => new Product(
    p.Id, request.Name, p.Price
));
```

5. PagedResult Support

MapFlow includes a built-in PagedResult<T> class for paginated responses.

```
PagedResult<Product> paged = await repository.GetAllAsync(...);

PagedResult<ProductDto> dtos = paged.Map(p =>
    new ProductDto(p.Id, p.Name, p.Price));

return Ok(dtos);
```

6. Source Generator Deep Dive

The MapFlow Source Generator uses Roslyn's incremental generator API, which means it only re-runs when source files change, not on every keystroke. This keeps edit-and-continue and IDE responsiveness fast.

What the SG detects

- Classes, structs, records, record structs implementing IMapFrom<T> or IMapTo<T>
- Partial types (the generated code merges with your partial declarations)
- Both implicit and explicit interface implementations
- Generic constraint clauses on the type parameter
- Property name and type matching between source and destination

What the SG generates

For a type like ProductDto : IMapFrom<Product>:

```
// Auto-generated by MapFlow.SourceGenerator
partial class ProductDto
{
    void IMapFrom<Product>.MapFrom(Product source)
    {
        this.Id = source.Id;
        this.Name = source.Name;
        this.Price = source.Price;
        CustomMapFrom(source);
    }
    partial void CustomMapFrom(Product source);
}
```

The generator uses explicit interface implementation to avoid polluting your public API. The CustomMapFrom partial method is a no-op by default.

7. API Reference

Mapper Static Class

Method	Returns	Description
Map<T,S>(T)	S	Interface-based single mapping
Map<T,S>(IEnumerable)	List<S>	Interface-based collection
Apply<T>(T, Action)	T	Mutate and return same instance
Apply<T>(T, Func)	T	Transform and return new instance

MapperExtensions (Fluent)

Method	Returns	Description
source.Map(sel)	TDest	Selector-based single mapping
source.Map(sel) collectior	List<TDest>	Selector-based collection
source.MapTo<T>()	T	Fluent interface-based mapping
source.Apply(Action)	TSource	Mutate and return same
source.Apply(Func)	TSource	Transform and return

PagedResult<T> Instance Methods

Method	Returns	Description
Map<TDest>(Func)	PagedResult<TDest>	Project items preserving page metadata

Interfaces

Interface	Method	Description
IMapFrom<T>	void MapFrom(T src)	Source -> Target mapping
IMapTo<T>	T MapTo()	Target -> Source reverse

8. Null Safety & Argument Validation

Every public entry point in MapFlow validates its arguments and throws ArgumentNullException with the parameter name when null is passed.

- Mapper.Map(source, selector) validates both source and selector
- Mapper.Apply(source, mutator) validates both
- source.Map(selector) validates both source and selector
- source.MapTo<T>() validates source
- source.Apply(mutator) validates both
- PagedResult<T>.Map(selector) validates selector

No silent NullReferenceExceptions. Pass null, get an immediate clear exception.

9. Performance Characteristics

All modes share one property: there is no warmup cost. Every mode is fast from the first call.

Mode	First call	10K calls	Startup
Lambda	~0 ns	~0 ns overhead	None
Interface (manual)	~10 ns	~10 ns	None
Source Generator	~10 ns	~10 ns	None
AutoMapper (ref)	~500 ms	~20 ns	Seconds
Mapster (ref)	~100 ms	~15 ns	~500ms

Reference values assume typical DTO projections with 5-10 properties. Lambda mode is the same cost as writing the assignment by hand because it IS the same IL. Interface and SG modes add a single virtual call overhead beyond hand-written code, which is negligible.

10. AOT Compatibility

MapFlow is fully compatible with .NET Native AOT (PublishAot). This is not an accident - it is a design requirement.

Why most mappers fail AOT

Native AOT compiles everything ahead of time. The runtime cannot generate or compile IL code. AutoMapper and Mapster depend on:

- System.Reflection.Emit - to create dynamic assemblies at runtime
- Expression<T>.Compile() - to compile expression trees into delegates
- PropertyInfo.SetValue/GetValue - to copy values via reflection

None of these APIs work under Native AOT.

What MapFlow does instead

Lambda mode uses your own delegates, compiled into the native image like any other code. Interface mode calls your methods directly. Source Generator mode produces C# code at build time that becomes part of the native image.

```
# Publish with AOT - just works
dotnet publish -c Release -r win-x64 --self-contained -p:PublishAot=true
```

11. Comparison with AutoMapper & Mapster

Aspect	AutoMapper / Mapster	MapFlow
Reflection	Used for config + mapping	Zero reflection
Dependencies	Many transitive deps	Zero dependencies
AOT compatible	No	Yes - all modes
Startup cost	Seconds (large projects)	Zero
Memory overhead	High (expression cache)	None
DI required	Usually	Not required
Convention magic	Yes (naming rules)	No (explicit only)
Source Generator	Experimental / add-on	Built-in, first class
Null safety	NRE on bad config	ArgumentNullException
Debugging	Stack trace through emit	Your code or interfaces
Learning curve	Medium-high (profiles)	Low (interfaces)

When AutoMapper/Mapster make sense: large legacy codebases already using them, or when you need advanced convention-based flattening.

When MapFlow makes sense: greenfield projects, AOT-targeted applications, projects that value startup time, or teams that prefer explicit over magic.

12. Supported Types

Type	Notes
class	Standard reference types
record	Compiler-generated equality + deconstruction
record struct	Value type records
struct	Value types via generic constraints
readonly struct	Immutable value types
partial class/struct	Required for SG integration

The SG detects type kind automatically using Roslyn syntax kind checks. No manual flag or attribute needed.

13. Best Practices

1. Prefer Source Generator for DTOs

If your DTO has matching property names with the source entity, let the SG do the work. It generates the same code you would write by hand.

2. Use lambdas for one-off projections

When a mapping is specific to a single use case (e.g., a report projection), a lambda selector is the clearest and fastest option.

3. Use Apply() for updates

When modifying existing entities, Apply() with an Action<T> is cleaner than creating temporary variables and enables method chaining.

4. Use CustomMapFrom for edge cases

When the SG auto-maps most properties but you need custom logic for some, implement CustomMapFrom() rather than writing the entire MapFrom() manually.

5. Keep DTOs partial for SG compatibility

The SG requires partial types. Always declare your DTOs as partial classes when using IMapFrom or IMapTo with auto-generation.

14. Migration Guide

From a custom MapperConfiguration

If you have a static helper class with Map() and Apply() extensions:

```
// BEFORE:
public static class MapperConfiguration {
    public static TDestination Map<T,TD>(this T s, Func<T,TD> sel)
        => sel(s);
}

// AFTER: delete MapperConfiguration.cs
// Add <Using Include="MapFlow" /> in csproj
// MapFlow provides the same extension methods with null guards
```

From AutoMapper

```
// BEFORE (AutoMapper):
var config = new MapperConfiguration(cfg => {
    cfg.CreateMap<Product, ProductDto>();
});
var dto = mapper.Map<ProductDto>(product);

// AFTER (MapFlow):
var dto = product.Map(p => new ProductDto(
    p.Id, p.Name, p.Price));

// For repeatable mappings:
public partial class ProductDto : IMapFrom<Product> { }
var dto = Mapper.Map<Product, ProductDto>(product);
```

15. FAQ

Q: Does MapFlow support nested object mappings?

A: MapFlow does not auto-map nested objects by convention. For nested mappings, use a lambda selector or manually map in CustomMapFrom(). This is intentional - nested auto-mapping is a common source of hidden behavior and performance surprises.

Q: Can I use MapFlow with dependency injection?

A: Yes, but you don't need to. Mapper.Map<T> is a static method. There is no required DI setup or service registration.

Q: Does MapFlow support IQueryable projections?

A: Not directly. MapFlow is for in-memory object mapping. For IQueryable, use Select() directly to let EF translate to SQL.

Q: Can MapFlow map to an existing object?

A: Yes. Use Mapper.Map(source, destination) or Apply(transform).

Q: Does the SG work with sealed types?

A: Yes. Sealed is fine, the SG only needs partial on the target type.

Q: Is there a perf difference between IMapFrom and IMapTo?

A: No. Both are single virtual calls. The direction is symmetric.

Q: Can I use MapFlow in Blazor WebAssembly?

A: Yes, especially beneficial because WASM AOT is fully supported.

Q: What about MAUI or Unity?

A: MapFlow targets net8.0, SG targets netstandard2.0. Works anywhere those frameworks run.

Q: How do I debug SG-generated code?

A: See the generated files in Dependencies > Analyzers > MapFlow.SourceGenerator in your project.

Q: Does MapFlow handle property renaming?

A: Not with attributes yet (planned). For now, use CustomMapFrom() to map properties with different names.